UNIVERSIDAD PRIVADA DEL VALLE

Ingeniería en Sistemas Informáticos



Tecnologías Emergentes I

INVESTIGACIÓN Y DESPLIEGUE DE UN REPOSITORIO BIG DATA CON DOCKER

Nombre : Bricely Clariss Gutierrez Cadima

Jhon Arnold Pari Siles

Benjamin Espinoza Vallejos

Grupo: A

2026

1. Objetivo

Implementar un entorno distribuido Hadoop HDFS compuesto por 1 NameNode y 3 DataNodes, configurando un factor de replicación mínimo de 2, para analizar experimentalmente el comportamiento de HDFS ante la caída de un DataNode mediante la carga y recuperación de datasets de 1 GB y 10 GB.

El estudiante deberá demostrar, mediante evidencias verificables, la distribución de los bloques, las réplicas existentes y el comportamiento del sistema antes, durante y después de la caída de un DataNode.

---

2. Requerimientos mínimos

Cada grupo deberá implementar un entorno con:

Componente	**Cantidad mínima**
NameNode	1
DataNode	3
Factor de replicación	≥ 2
Dataset 1	1 GB
Dataset 2	10 GB
Pruebas de fallo	2
Evidencias	Obligatorio

.

3. Identificación del grupo

Para garantizar que la práctica fue realizada por el grupo y no simplemente reproducida de otra implementación, **todos los directorios relacionados con la práctica deberán utilizar el número de grupo**.

Por ejemplo, si el grupo es el **Grupo 05**:

/grupo05/

Y dentro:

/grupo05/dataset_1GB/

/grupo05/dataset_10GB/

/grupo05/resultados/

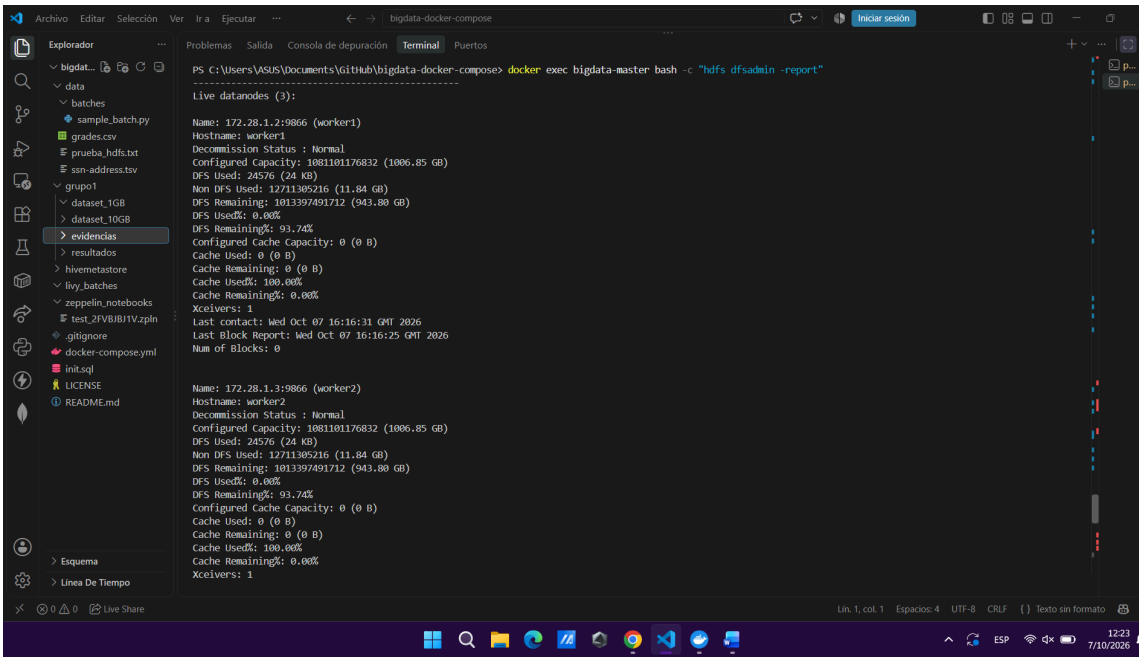
/grupo05/evidencias/

También deberán utilizarse nombres identificables para los archivos:

dataset_grupo05_1GB

dataset_grupo05_10GB

El número de grupo deberá aparecer además en las evidencias solicitadas.



---

4. Prueba de fallo con cada dataset (con la de 1GB y con la e 10GB)

Esta será una de las partes **más importantes de la práctica**.

Antes de detener un DataNode, deberán registrar el estado inicial.

Estado inicial

Por ejemplo:

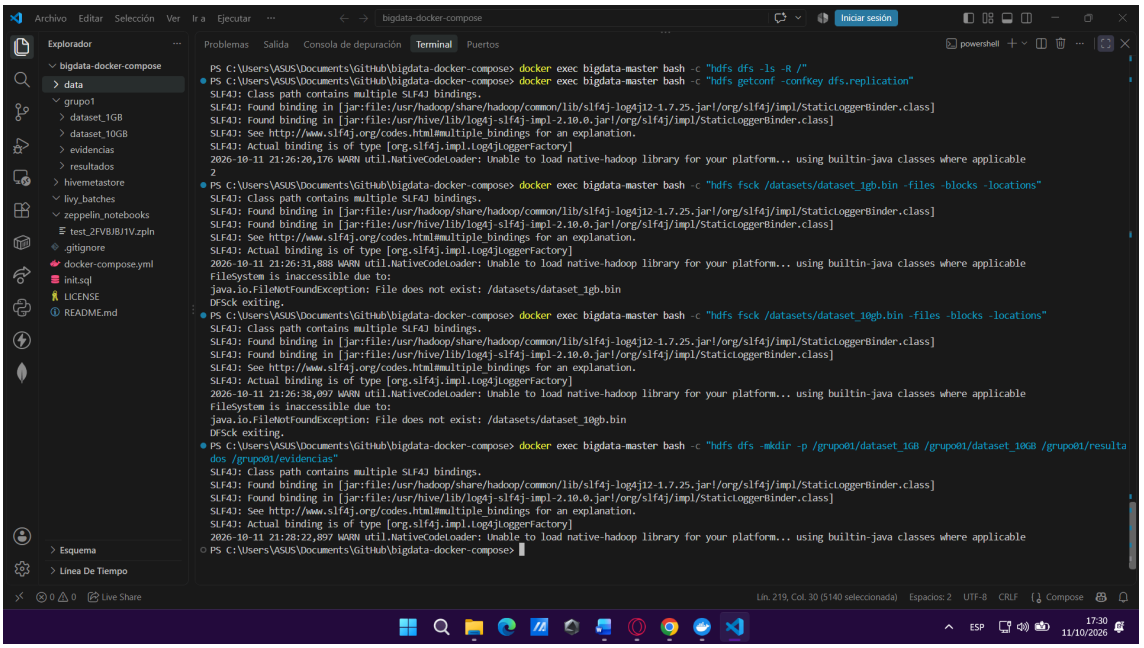
DN1 → Activo

DN2 ⟶ Activo

DN3 ⟶ Activo

Y deberán identificar dónde se encuentran las réplicas del dataset.

Primero como todavía no se subió los archivos alo que seriael hdfs este comando primero **docker exec bigdata-master bash -c "hdfs dfs -mkdir -p /grupo01/dataset_1GB /grupo01/dataset_10GB /grupo01/resultados /grupo01/evidencias"** el cual estaría creando una copia de esos archivos pero en hdfs



Luego se Crea y se sube el dataset de 1 GB los siguientes comandos

Crear un archivo de 1 GiB dentro del contenedor principal:

docker exec bigdata-master bash -c "dd if=/dev/zero of=/tmp/dataset_grupo01_1GB bs=1M count=1024 status=progress"

Subir el archivo a HDFS:

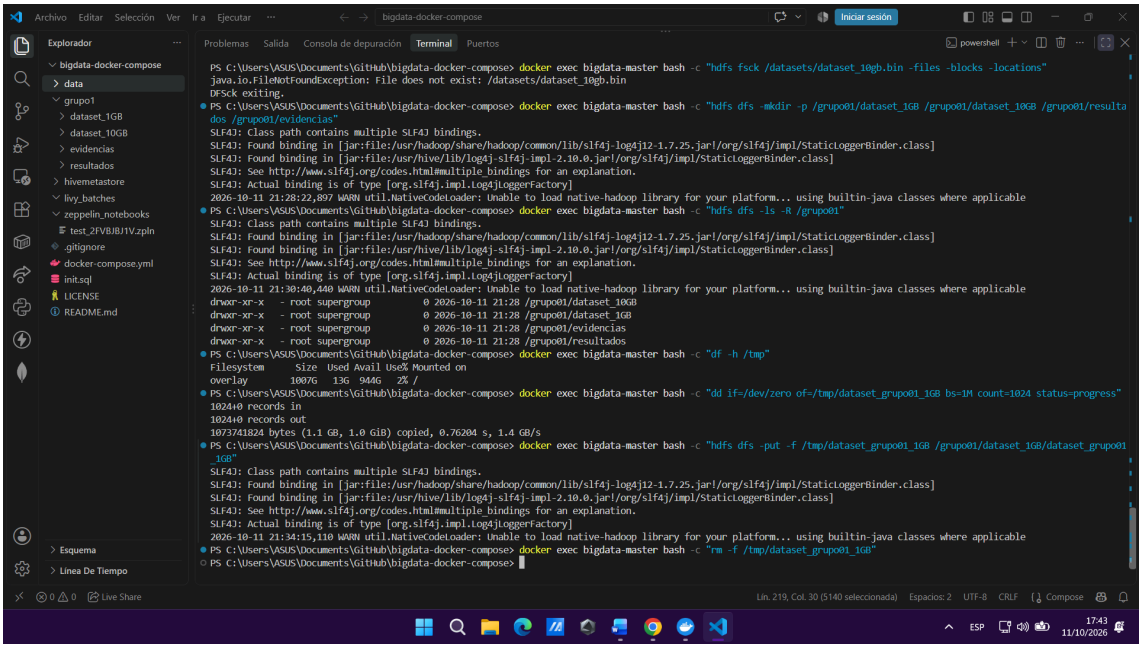
docker exec bigdata-master bash -c "hdfs dfs -put -f /tmp/dataset_grupo01_1GB /grupo01/dataset_1GB/dataset_grupo01_1GB"

comprobar el archivo:

docker exec bigdata-master bash -c "hdfs dfs -ls -h /grupo01/dataset_1GB"

Eliminar la copia temporal del contenedor para liberar espacio; el archivo de HDFS no se elimina:

docker exec bigdata-master bash -c "rm -f /tmp/dataset_grupo01_1GB"



Lo mismo para el de 10gb

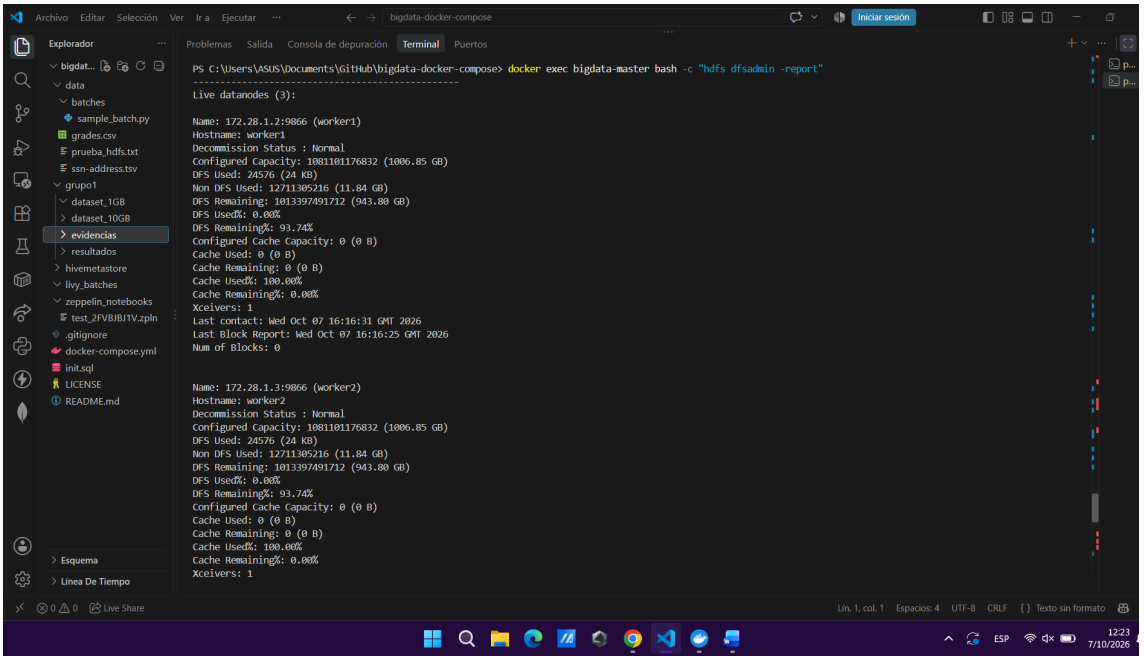
docker exec bigdata-master bash -c "dd if=/dev/zero of=/tmp/dataset_grupo01_10GB bs=1M count=10240 status=progress"

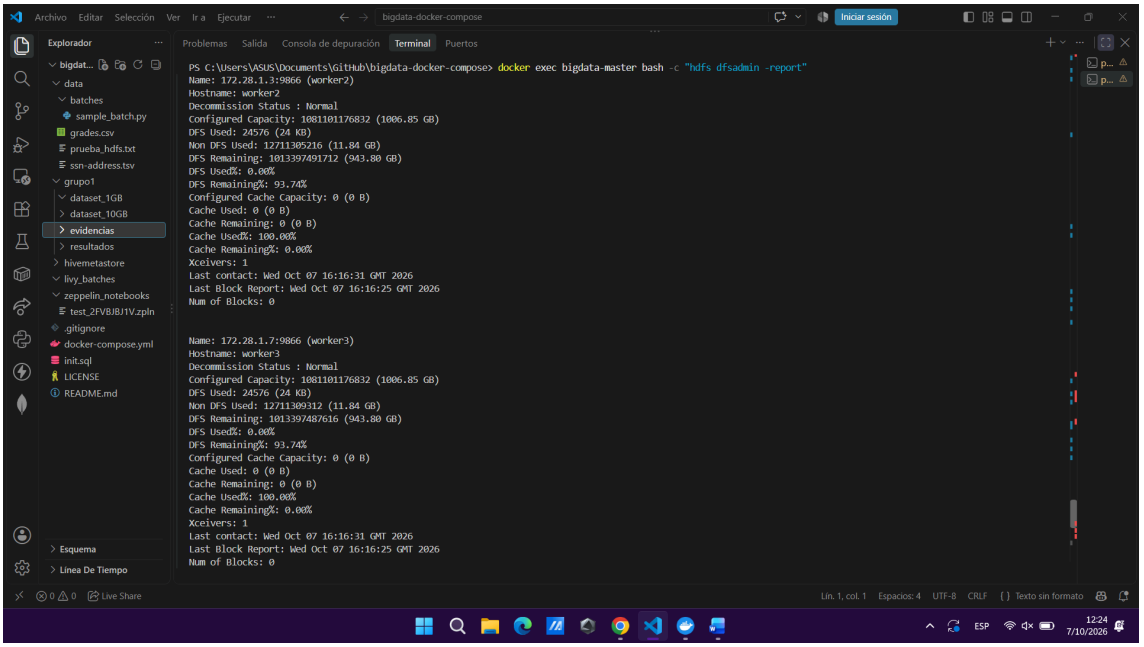
docker exec bigdata-master bash -c "hdfs dfs -put -f /tmp/dataset_grupo01_10GB /grupo01/dataset_10GB/dataset_grupo01_10GB"

docker exec bigdata-master bash -c "hdfs dfs -ls -h /grupo01/dataset_10GB"

docker exec bigdata-master bash -c "rm -f /tmp/dataset_grupo01_1GB"

Para esto se utilizo lo que seria el comando **docker exec bigdata-master bash -c "hdfs dfsadmin -report"**





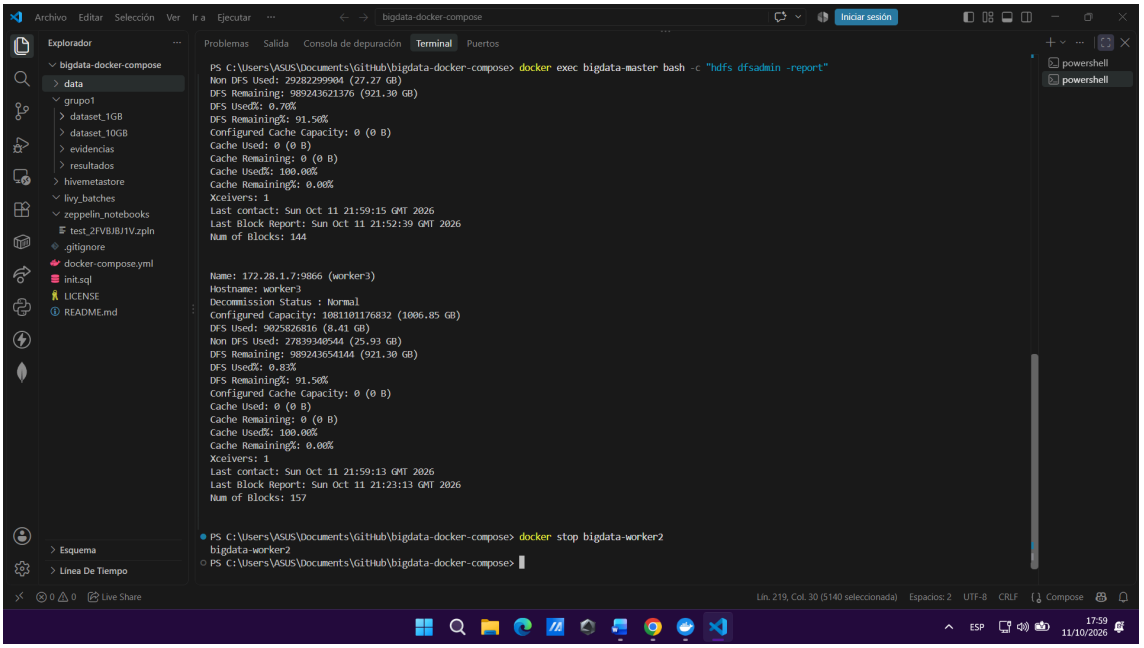
Posteriormente, deberán **detener solamente un DataNode**.

Por ejemplo:

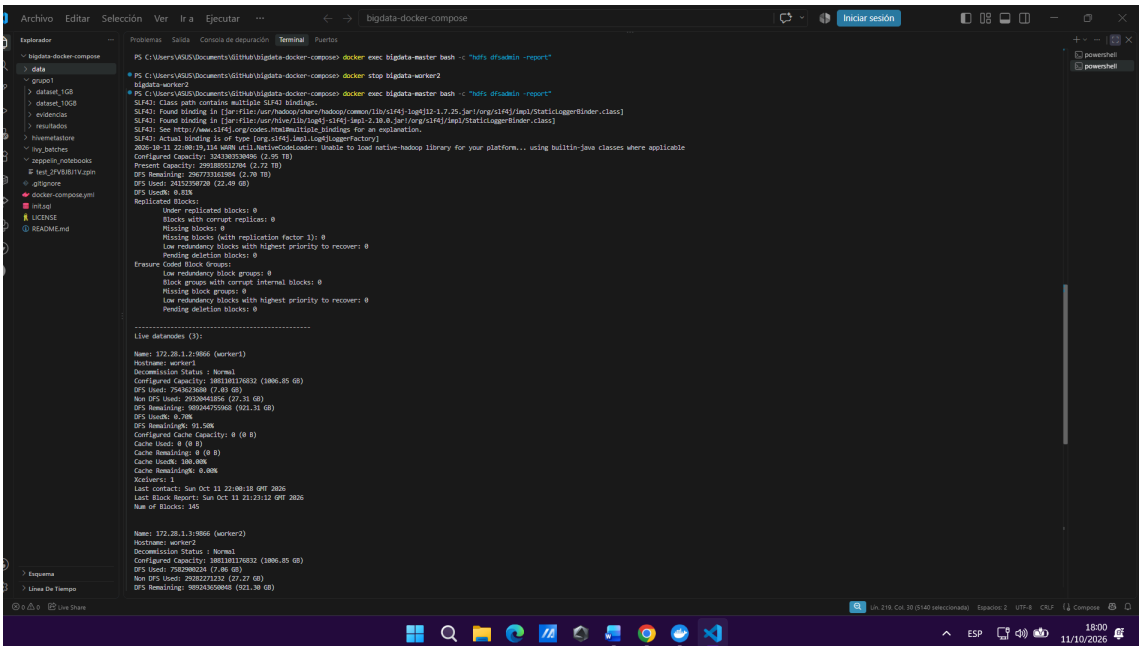
DN2 → FALLA

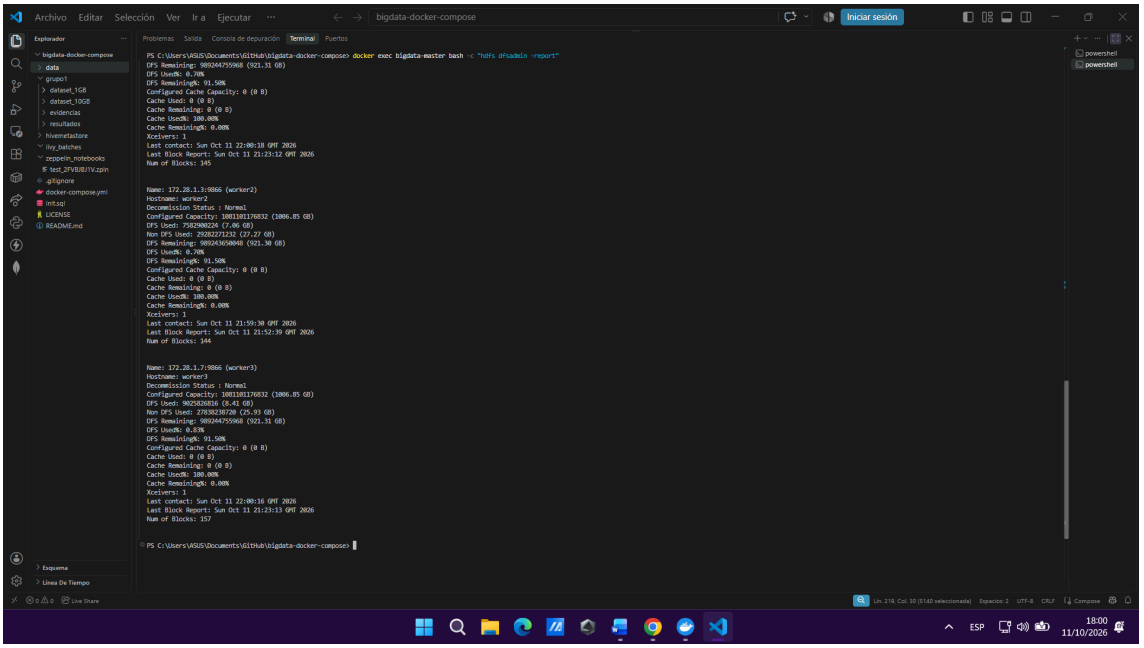
Primero se iso **docker exec bigdata-master bash -c "hdfs dfsadmin -report**" para ver si los datanode funcionan

Después se iso **docker stop bigdata-worker2** para detener el datanode 2



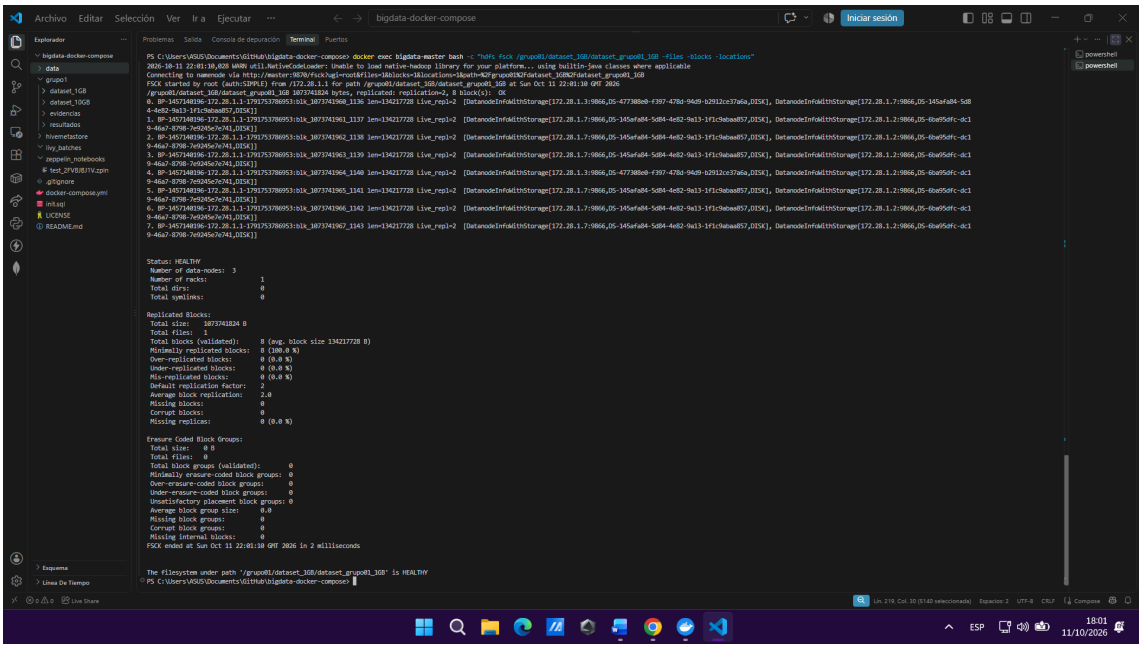
Luego se iso denuevo un report para saber si funcionan los 3 nodes y se puede ver que si siguen funcionando





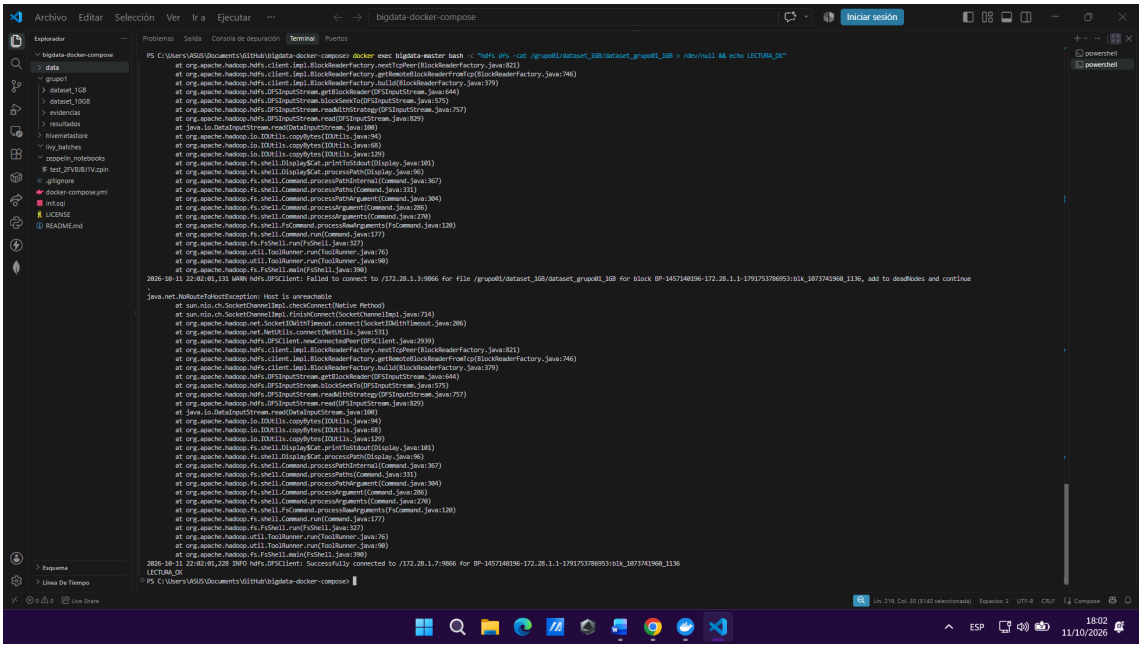
Después se iso la consulta los bloques del dataset de 1 GB

docker exec bigdata-master bash -c "hdfs fsck /grupo01/dataset_1GB/dataset_grupo01_1GB -files -blocks -locations"



Finalmente, se iso la comprobación de que todavía sea posible leer el archivo completo sin mostrar su contenido en la terminal

docker exec bigdata-master bash -c "hdfs dfs -cat /grupo01/dataset_1GB/dataset_grupo01_1GB > /dev/null && echo LECTURA_OK"



Deberán demostrar

¿Qué ocurre con el dataset de 1 GB?

Deberán verificar:

- Estado del DataNode.
- Estado del archivo.
- Estado de los bloques.
- Número de réplicas disponibles.
- Si el archivo continúa siendo accesible.
- Si se puede leer el archivo.
- Si HDFS identifica bloques subreplicados.
- Si HDFS inicia o no un proceso de replicación.

Por ejemplo:

hdfs dfsadmin -report

y:

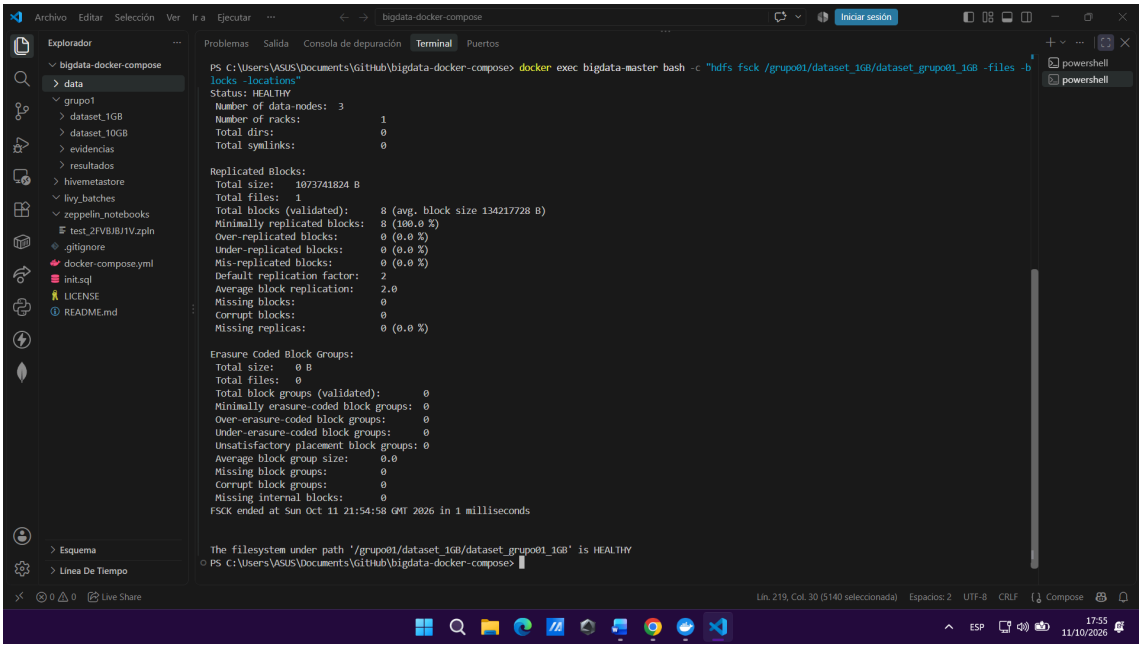
hdfs fsck /grupo05/dataset_1GB -files -blocks -locations

para ver este reporte :

docker exec bigdata-master bash -c "hdfs fsck /grupo01/dataset_1GB/dataset_grupo01_1GB -files -blocks -locations"

para ver el dataset de 10gb:

docker exec bigdata-master bash -c "hdfs fsck /grupo01/dataset_10GB/dataset_grupo01_10GB -files -blocks -locations"



---

5. Análisis

5. Análisis de tolerancia a fallos en HDFS

Para esta parte del informe de Grupo 1, hay que distinguir entre los resultados que observaste realmente en tu clúster y la explicación teórica de HDFS. Los números exactos de bloques, sus DataNodes y las nuevas réplicas deben salir de tus comandos hdfs fsck y hdfs dfsadmin -report; no conviene inventarlos.

5.1. Antes, durante y después de la falla

Puedes utilizar estas tablas en tu informe y completar los datos con las evidencias de las pruebas de los datasets de 1 GB y 10 GB.

A. Antes de la falla

Aspecto	Qué debes registrar
Número de bloques	El total indicado por hdfs fsck para el archivo.

Réplicas por bloque	Lo esperado es 2, porque dfs.replication = 2; comprueba el estado real de cada bloque.
DataNodes que almacenan los bloques	Los nombres o direcciones que aparecen en hdfs fsck -files -blocks -locations.
Estado del clúster	Los DataNodes activos que muestra hdfs dfsadmin -report.

B. Durante la falla

Pregunta	**Qué debes registrar**
¿Qué DataNode se detuvo?	El nombre exacto del contenedor detenido, por ejemplo, bigdata-worker2, si ese fue el que utilizaste.
¿Qué bloques estaban almacenados allí?	Los bloques cuya ubicación incluía ese DataNode en la evidencia inicial.
¿Existían otras réplicas?	Comprueba si cada bloque afectado conservaba otra copia en un DataNode activo.
¿Continuó disponible el archivo?	Comprueba si HDFS permite consultar y leer el archivo durante la falla.
¿Qué cambió en HDFS?	El DataNode detenido deja de estar disponible tras detectarse la pérdida de sus latidos (heartbeats). Puede haber bloques subreplicados temporalmente.

C. Después de la falla

Aspecto	**Qué debes registrar**
Estado de HDFS	Número de DataNodes activos y capacidad disponible, mediante hdfs dfsadmin -report.
Bloques subreplicados	Los bloques con menos réplicas de las solicitadas, según hdfs fsck.
Recuperación automática	Si HDFS comienza a copiar bloques para recuperar el factor de replicación.
Ubicación de nuevas réplicas	El DataNode que recibe cada nueva copia, comprobándolo con otra consulta hdfs fsck -files -blocks -locations.

Importante: la recuperación no siempre es inmediata. El NameNode primero debe detectar que el DataNode dejó de responder; después programa la replicación de los bloques afectados. Si solo detienes un DataNode y quedan otros dos activos,

normalmente HDFS puede reconstruir la segunda réplica en uno de los nodos restantes, siempre que haya espacio y el clúster pueda realizar la operación.

5.2. Explicación de los resultados

HDFS utiliza un mecanismo de replicación para tolerar fallos. El archivo se divide en bloques y cada bloque se almacena en varios DataNodes según el factor de replicación configurado.

En nuestro clúster, el factor de replicación es 2, por lo que HDFS intenta mantener dos réplicas de cada bloque. Si un DataNode falla, las otras copias permiten continuar accediendo a los datos. El NameNode detecta la pérdida y puede ordenar la creación de nuevas réplicas para recuperar el nivel de protección configurado.

Preguntas para la exposición

Pregunta 1. ¿Por qué HDFS puede continuar proporcionando el archivo aunque un DataNode haya fallado?

Porque HDFS almacena varias réplicas de cada bloque en diferentes DataNodes. Si uno deja de funcionar, el cliente puede leer las copias que permanecen en los nodos activos. Así, la falla de un nodo no implica necesariamente la pérdida o indisponibilidad del archivo completo.

Pregunta 2. ¿Qué habría ocurrido si el factor de replicación hubiera sido 1?

Cada bloque tendría una sola réplica. Si fallara el DataNode que contiene un bloque y no existiera otra copia recuperable, ese bloque dejaría de estar disponible. Si el archivo contiene bloques inaccesibles, no podría leerse completamente hasta recuperar los datos.

Pregunta 3. ¿Qué habría ocurrido si hubieran fallado dos DataNodes simultáneamente?

Con tres DataNodes y un factor de replicación de 2, algunos bloques podrían conservar una copia en el único DataNode restante, mientras que otros podrían perder todas sus copias accesibles. El resultado dependería de dónde estuvieran distribuidas las réplicas de cada bloque. Por eso, HDFS con replicación 2 no garantiza que el archivo siga completamente disponible ante cualquier combinación de dos fallos.

Pregunta 4. ¿Por qué un factor de replicación de 2 no significa necesariamente que existan dos copias completas del archivo en dos DataNodes?

Porque la replicación se aplica a cada bloque individual, no al archivo completo como una única unidad. Los bloques se distribuyen entre los DataNodes según las decisiones de colocación de HDFS. Por tanto, las dos réplicas de distintos bloques pueden estar distribuidas de manera diferente entre los nodos.

Pregunta 5. ¿Cuál es la relación entre tamaño de bloque y número de bloques?

El número de bloques depende del tamaño del archivo y del tamaño configurado para cada bloque. Si el archivo es más grande o el tamaño de bloque es menor, se necesitan más bloques.

Por ejemplo, suponiendo bloques de 128 MiB y tamaños binarios:

Tamaño del archivo	Número aproximado de bloques
1 GiB	8
10 GiB	80

Estos números son ejemplos: debes verificar el tamaño de bloque real de tu clúster. El último bloque normalmente puede ser más pequeño que los demás.

Pregunta 6. ¿Por qué el dataset de 10 GB permite observar un comportamiento más representativo de un sistema distribuido que el dataset de 1 GB?

Porque normalmente contiene más bloques y permite observar una distribución más amplia de estos entre los DataNodes. Esto facilita analizar más ubicaciones de réplicas, detectar bloques subreplicados y observar la recuperación tras una falla. Sin embargo, un archivo de 10 GB no garantiza por sí solo una distribución diferente: esta también depende del tamaño de bloque y de las decisiones del NameNode.

Pregunta 7. ¿Qué sucede con los bloques que tenían una réplica en el DataNode que falló?

Si un bloque tenía dos réplicas y una estaba en el nodo que falló, puede quedar temporalmente con una sola réplica accesible. El NameNode detecta la pérdida, marca el bloque como subreplicado y programa la creación de otra copia desde una réplica disponible, si las condiciones del clúster lo permiten.

Pregunta 8. ¿Quién administra la ubicación y replicación de los bloques: el cliente, el DataNode o el NameNode?

El NameNode administra los metadatos del sistema de archivos, registra las ubicaciones conocidas de los bloques y coordina la replicación. Los DataNodes almacenan y sirven los bloques, mientras que el cliente solicita la información de ubicación al NameNode y se comunica con los DataNodes para leer o escribir datos.

Pregunta 9. ¿Qué diferencia existe entre disponibilidad del archivo y cumplimiento del factor de replicación después de una falla?

La disponibilidad significa que los datos necesarios para leer el archivo siguen accesibles. El cumplimiento del factor de replicación significa que cada bloque cuenta con el número de réplicas solicitado.

Un archivo puede seguir disponible aunque algunos bloques tengan una sola réplica accesible, cuando el factor configurado es 2. En ese caso, el archivo se puede leer, pero su protección frente a nuevos fallos es menor hasta que HDFS recupere las réplicas faltantes.

---

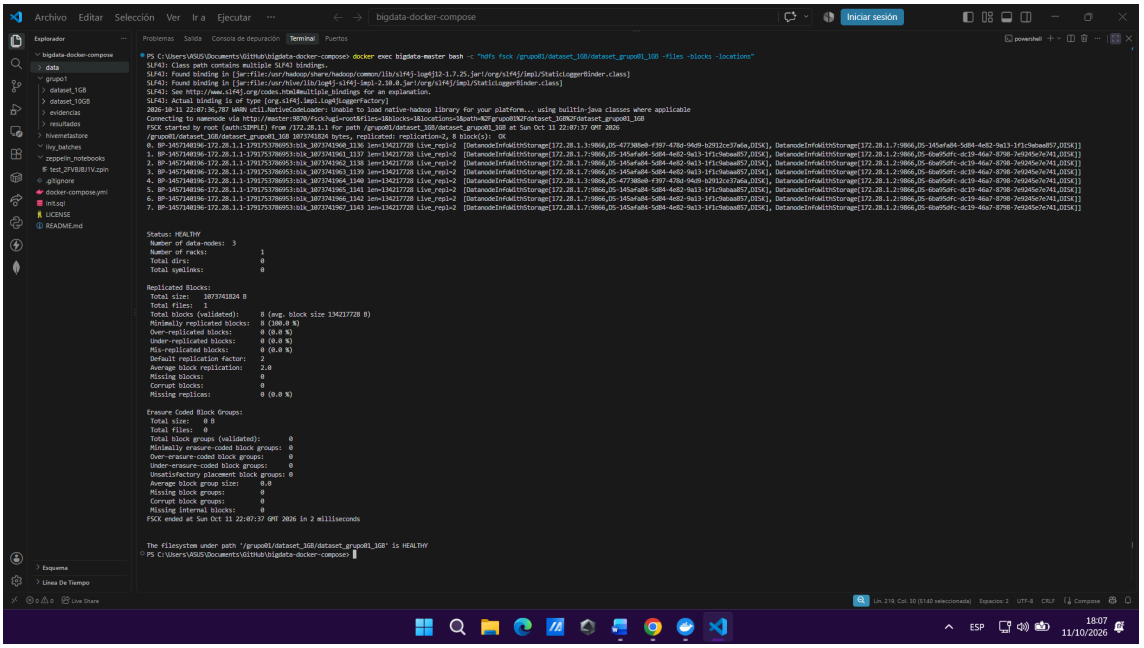
6. Tabla comparativa

Puedes copiar esta tabla en tu informe de Grupo 1. Las celdas marcadas como pendiente se completan con los resultados reales de las pruebas de fallo.

Característica	Dataset 1 GB	Dataset 10 GB
Tamaño	1 GiB (1.073.741.824 bytes)	10 GiB (10.737.418.240 bytes)
Número de bloques	8	80
Factor de replicación	2	2
DataNode afectado	Pendiente: el que detuviste en la prueba de 1 GB	Pendiente: el que detuviste en la prueba de 10 GB
Bloques afectados	Pendiente: verificar con hdfs fsck	Pendiente: verificar con hdfs fsck
Bloques disponibles después de la falla	Pendiente de verificar	Pendiente de verificar
¿Archivo accesible?	Pendiente: comprobar si apareció LECTURA_OK	Pendiente: comprobar si apareció LECTURA_OK
¿Existieron bloques subreplicados?	Pendiente: revisar el resultado de fsck	Pendiente: revisar el resultado de fsck
Tiempo de recuperación	Pendiente de medir	Pendiente de medir
Nuevas réplicas	Pendiente: comprobar si Hadoop las generó automáticamente	Pendiente: comprobar si Hadoop las generó automáticamente

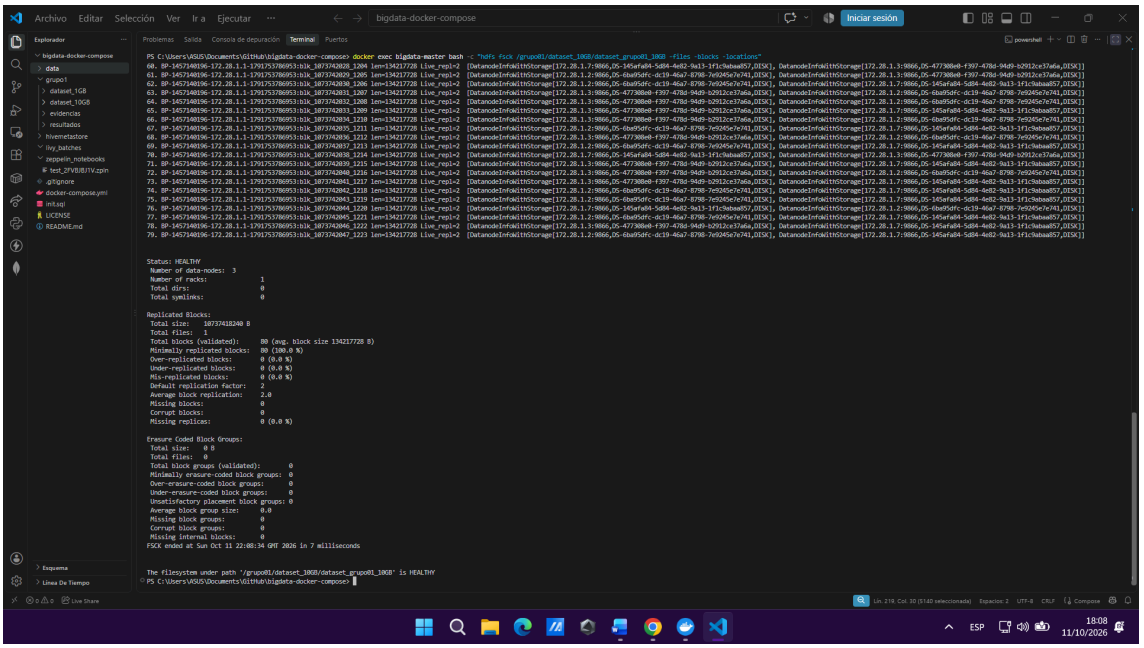
Para el dataset de 1 GB:

docker exec bigdata-master bash -c "hdfs fsck /grupo01/dataset_1GB/dataset_grupo01_1GB -files -blocks -locations"



Para el dataset de 10 GB:

docker exec bigdata-master bash -c "hdfs fsck /grupo01/dataset_10GB/dataset_grupo01_10GB -files -blocks -locations"



CONCLUSION

Comparación de los experimentos

Los experimentos permiten comparar el comportamiento de HDFS ante la falla de un DataNode utilizando dos datasets de diferente tamaño. El archivo de 1 GB está distribuido en 8 bloques, mientras que el archivo de 10 GB está distribuido en 80

bloques. Ambos utilizan un factor de replicación de 2, por lo que cada bloque cuenta inicialmente con dos copias.

Se espera que HDFS mantenga la disponibilidad de los archivos cuando falla un DataNode, siempre que exista al menos una réplica accesible de cada bloque. El NameNode detecta la pérdida del nodo y puede ordenar la creación de nuevas réplicas para recuperar el nivel de replicación configurado. La comparación de los resultados permitirá determinar si los archivos continuaron accesibles, cuántos bloques quedaron temporalmente sub replicados y cuánto tardó el clúster en recuperarse.